claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_d9866476-800\agent-abb85caee45a6a1e7.jsonl`
- SHA-256: `8d0e1ff6ae79006662911bb2e600a45aa9df13b43a7fec7e066564f1be9a96a8`
- Source modified: `2026-06-22T09:13:57+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d9866476-800`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:12:06.301Z)

```
You are mapping the badminton-highlight-indexer repo, checked out at origin/master under
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files
under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal
is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short
golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw
structured data, not prose for a human.

ADVERSARIALLY VERIFY this claim ? try to REFUTE it with concrete code evidence under
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e; default to
"refuted"/"uncertain" if the evidence isn't clearly there. Claim:
"The COUNT of reels/highlight-segments produced by the configured+checkpointed pipeline is
DETERMINISTIC for a fixed (video, config, weights) ? safe to assert exactly (==), not just
approximately."

Context from the mapping phase (may be wrong ? re-check against code):
[
 {
  "summary": "\nMapped the reel-production path end-to-end in the badminton-highlight-indexer.
The regression test signal is: **COUNT OF PLAY_SEGMENTS ROWS in the SQLite database per video
after segmentation + filtering**. One stitched output reel (ONE .mp4 file per /api/compile call)
is created from N filtered play_segments. \n\nThe PIPELINE FLOW:\n1. /api/process (or CLI
--video-path) ? _execute_processing ? video_id = MD5(video_file)\n2. Segmenter.process_video ?
detects rally intervals ? inserts rows into play_segments table\n3. /api/compile receives
segment_ids ? queries play_segments ? applies stitching.min_shot_count filter ? calls
VideoCompiler.compile_highlights with (start_time, end_time) pairs\n4. VideoCompiler trims each
segment (with begin/end padding), re-encodes with optional watermark, ffmpeg concat demuxer
stitches N trimmed clips ? ONE .mp4 reel output file\n\n**Regression signal:** `SELECT COUNT(*)
FROM play_segments WHERE video_id = ?` after segmentation completes. This is stable,
deterministic, and queryable headlessly via the database.\n",
  "key_facts": [
   "video_id = compute_video_id(local_path) = MD5 hash
(backend/utils/hashing.py:compute_video_id)",
   "play_segments table schema: id (AUTOINCREMENT), video_id, start_time, end_time, duration,
server, receiver, metadata (JSON). Rows inserted by segmenter via db.add_play_segment().",
   "Segmentation path: main.py::_execute_processing (line 439) ? segmenter.process_video() ?
returns list of rally dicts ? each inserted as play_segments row via
db_instance.add_play_segment()",
   "Compilation path: /api/compile (line 705) ? query_play_segments(video_id, {}) ? filter by
segment_ids + stitching.min_shot_count (from metadata['shot_count']) ? extract time_pairs
[(start, end), ...] ? VideoCompiler.compile_highlights(video_path, time_pairs,
output_filename)",
   "VideoCompiler.compile_highlights (compiler.py line 305): normalizes timestamps ? merges
overlapping ranges ? for each segment: _trim_one_segment() (line 192) ffmpeg -ss -to re-encodes
with padding + watermark ? ffmpeg concat demuxer stitches clips ? ONE output .mp4 file",
```

```
   "Stitching config (backend/config/models.py line 255): begin_padding=0.5s, end_padding=0.5s
(per-segment), min_shot_count=1 (metadata filter), watermark enabled by default",
   "Output: ONE .mp4 file per /api/compile call at output_dir/{output_filename}. Stored flat (no
video_id subdirs) ? /api/reels lists all .mp4 files in output_dir.",
   "Regression signal (stable, headless, deterministic): `SELECT COUNT(*) FROM play_segments
WHERE video_id = <video_id>` after _execute_processing completes (after destroy-AFTER-confirm
finalization at main.py line 536).",
   "Database path: global_config.output.output_dir / global_config.output.db_name (default:
output/rally_indexer.db)",
   "CLI single-shot headless mode: `indexer --video-path <path> [--segmenter <name>] [--output-
dir <dir>]` (no web UI, exits after segmentation) ? main.py line 997-1075"
  ],
  "recommendation": "\nFor the NIGHTLY E2E REGRESSION on 2?3 golden videos:\n\n**REGRESSION
SIGNAL:** Count of play_segments rows in the output DB for each golden video. Expected counts
should be hardcoded from HEAD baseline.\n\n**EXACT HEADLESS COMMANDS:**\n\n```bash\n# 1. Run
segmentation (single-shot CLI mode, no server, no UI):\nindexer --video-path
/path/to/golden_video.mp4 \\\n        --segmenter motion \\\n        --output-dir
output/nightly_e2e\n\n# 2. Query the segment count (read-only DB inspection):\npython -c \"\n
from backend.infrastructure.database import Database\n  db =
Database('output/nightly_e2e/rally_indexer.db')\n  from backend.utils.hashing import
compute_video_id\n  video_id = compute_video_id('/path/to/golden_video.mp4')\n  with
db._connect() as conn:\n    count = conn.cursor().execute(\n      'SELECT COUNT(*) FROM
play_segments WHERE video_id = ?', \n      (video_id,)\n    ).fetchone()[0]\n
print(count)\n\"\n\n# 3. Assert count == baseline for each video:\n#    Fail (exit 1) if ANY
video's count deviates\n```\n\n**NIGHTLY REGRESSION HARNESS STRUCTURE:**\n\n1. **Checkpointed
baseline:** `eval_baselines/nightly_e2e_baselines.json` (commit into repo)\n   ```json\n   {\n
\"golden_videos\": [\n       {\"name\": \"video1.mp4\", \"md5\": \"<video_id>\",
\"expected_segment_count\": 3},\n       {\"name\": \"video2.mp4\", \"md5\": \"<video_id>\",
\"expected_segment_count\": 5},\n       {\"name\": \"video3.mp4\", \"md5\": \"<video_id>\",
\"expected_segment_count\": 4}\n     ],\n     \"generator\": \"HEAD <commit>\",\n
\"timestamp\": \"2026-06-22T00:00:00Z\"\n   }\n   ```\n\n2. **Nightly runner script:** Run
headlessly on each golden video ? count play_segments ? compare to baseline ? exit 1 on
mismatch\n\n3. **STITCHING PIPELINE VERIFICATION (optional, post-segmentation):**\n   - After
segment count passes, optionally call /api/compile via Python to produce a test reel\n   -
Assert output .mp4 file exists and has size > 0\n   - (Stitching is more complex ? depends on
segment timing + ffmpeg + watermark rendering; segment count is the stable proxy)\n\n**EXACT
INTEGER TO TRACK:**\n- **Primary:** `COUNT(*) FROM play_segments WHERE video_id = <video_id>`
after segmentation\n- **Secondary (optional):** COUNT of .mp4 files in output_dir that match the
compile request (should be 1 per /api/compile call)\n\n**CONFIG FOR GOLDEN VIDEOS:**\n- Use
`--segmenter motion` (offline, no API key, no GPU ? reproducible)\n- Default padding/watermark
(stitching.begin_padding=0.5, end_padding=0.5)\n- No cloud storage (use local paths only)\n",
  "confidence": "high",
  "code_refs": [
   "backend/main.py:439::_execute_processing",
   "backend/main.py:559::process_video",
   "backend/main.py:705::compile_highlights",
   "backend/main.py:997-1075",
   "backend/infrastructure/database.py:24::Database",
   "backend/infrastructure/database.py:564::query_play_segments",
   "backend/infrastructure/database.py:459::add_play_segment",
   "backend/pipelin
```


2. assistant (2026-06-22T09:12:08.636Z)

I'll adversarially verify this claim by reading the actual code and checking whether the count of play_segments is truly deterministic. Let me start by examining the codebase.

3. user (2026-06-22T09:12:10.065Z)

```
689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app
(#312)
f8b2c3b feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run
default (gate O8) (#310)
0ed4cc6 fix(segmenter): fail the job on an unrunnable config instead of a silent 0-rally success
```

```
(Finding B) (#311)
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING_PLAN.md) + index pointers
(#309)
```

4. assistant (2026-06-22T09:12:13.242Z)

Now let me examine the core segmentation and database code to check for non-determinism:

5. user (2026-06-22T09:12:15.426Z)

```
430             path = resolve_under_root(path, base)
431         except ValueError as e:
432             raise HTTPException(status_code=400, detail=str(e)) from e
433         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
434             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
435         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
436         return FileResponse(path, media_type=media, filename=name)
437
438
439     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
440         """The full hash ? validate ? segment ? report pipeline for one video.
441
442         This is the former synchronous /api/process body, unchanged in behavior, now run on
443         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
444         the sync endpoint used to return (UI compatibility); the per-video observability
445         report is still emitted in `finally:` and grafted as response["reports"].
446
447         `progress` is an optional callable(stage: str) used to surface coarse job progress.
448         Raises on unexpected internal errors ? the caller records those as a job failure
449         (after this function has already restored the pre-run segments, see below).
450         """
451         notify = progress or (lambda stage: None)
452
453         notify("hashing")
454         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
455         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
456         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
457         # consumers (hash / validators / segmenter) use the resolved local path.
458         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
459         video_id = compute_video_id(local_video)
460         was_reingest = db_instance.get_video(video_id) is not None
461
462         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
463         notify("validating")
464         logger.info(f"Running validations for {req.video_path}...")
465         val_results, val_context = registry.run_all_with_context(local_video, global_config)
466         failures = [r for r in val_results if not r.passed]
467
468         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
469         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
470         seg_name = req.segmenter or default_seg
471         segmenter_actual = None
472         segmentation_ran = False
473         response: Optional[Dict[str, Any]] = None
474         try:
475             if failures:
476                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
477                 # status; it no longer destroys the video's prior good segments.
478                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
```

```
for r in val_results])
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": "Video failed validation checks.",
                    "results": [r.model_dump() for r in val_results],
                }
                return response

            # 2. Run segmenter & indexer
            logger.info("[API] Video passed checks. Segmenting and indexing...")
            db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                # Submit-time validation already 400s unknown names; this is the defensive
                # in-worker path (e.g. config default pointing at an unregistered
segmenter).
                available = list(segmenter_registry.list_available().keys())
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            logger.info(f"[API] Using segmenter: {seg_name}")
            notify(f"segmenting ({seg_name})")
            segmenter_actual = seg_name
            # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
            # if the run yields segments; otherwise drop the new partial rows and keep the
old.
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
```

```
536             _finalize_reingest(video_id, segments, play_wm, cand_wm)
537
538             response = {
539                 "video_id": video_id,
540                 "status": "success",
541                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
542                 "results": [r.model_dump() for r in val_results],
543                 "play_segments": segments,
544             }
545             return response
546         finally:
547             # Always emit the per-video observability report (next to the video, or to
548             # report.output_dir). Never raises. Surface the paths in the response.
549             paths = emit_indexer_report(
```

6. user (2026-06-22T09:12:15.838Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer.
```

7. assistant (2026-06-22T09:12:17.827Z)

Let me check the segmentation directory structure and examine key segmentation files:

8. user (2026-06-22T09:12:19.344Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/base.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/fusion_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/gemini.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/motion.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/tracknet_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/trajectory_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/wasb_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/yolo_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/_keyhint.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/__init__.py
```

9. assistant (2026-06-22T09:12:22.839Z)

Let me examine the motion segmenter (the simplest/most deterministic one) and the base segmenter class:

10. user (2026-06-22T09:12:23.664Z)

```
1       import cv2
2       import time
3       import random
4       from typing import List, Dict, Any, Tuple
5       import logging
6       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7       from backend.results import Success, Failure  # standardized contract
8
9       logger = logging.getLogger(__name__)
```

```
10
11      class MotionPlaySegmenter(BasePlaySegmenter):
12          @property
13          def name(self) -> str:
14              return "motion"
15
16          @property
17          def description(self) -> str:
18              return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21              """
22              Processes a video to detect play segments using motion heuristics,
23              populates SQLite with detected segments, and indexes detailed events.
24              """
25              if not player_pool:
26                  player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28              cap = cv2.VideoCapture(video_path)
29              if not cap.isOpened():
30                  logger.error(f"Segmenter could not open video: {video_path}")
31                  return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33              final_segments = self._detect_motion_segments(cap, max_frames)
34
35              segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37              return Success(value=segments_data)
38
39          def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40              """Run the pixel-motion heuristic over ``cap`` and return the detected
41              (start, end) play segments.
42
43              Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44              thresholds it into raw segments, then merges dead-time gaps and filters
45              out segments shorter than ``min_segment_duration``. ``cap`` is released
46              before returning. This is the detection half of ``process_video`` ? it
47              performs no DB writes and no fabrication.
48              """
49              fps = cap.get(cv2.CAP_PROP_FPS)
50              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51              duration = total_frames / fps if fps > 0 else 0
52
53              # Sub-sample settings (5 FPS for analysis)
54              analysis_fps = 5.0
55              frame_interval = max(1, int(fps / analysis_fps))
56
57              idx_cfg = self.config.get("indexing", {})
58              motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59              min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60              dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62              prev_gray = None
63              motion_signals = []
64              timestamps = []
65
66              frame_idx = 0
67              processed_count = 0
68              t_start = time.time()
69              # Emit a progress line roughly every 5% of the video so long runs are
70              # observable instead of silent (a full 1080p match is ~tens of minutes).
```

```
71              progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72              next_progress = progress_step
73              logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74                          f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75              while True:
76                  # Skip decoding for intermediate frames using cap.grab() for high
performance
77                  for _ in range(frame_interval - 1):
78                      if not cap.grab():
79                          break
80                      frame_idx += 1
81
82                  ret, frame = cap.read()
83                  if not ret:
84                      break
85
86                  # Process frame
87                  gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
88                  # Apply Gaussian Blur to smooth noise
89                  gray = cv2.GaussianBlur(gray, (21, 21), 0)
90
91                  t = frame_idx / fps
92                  timestamps.append(t)
93
94                  if prev_gray is not None:
95                      # Frame absolute difference (highly optimized motion tracking)
96                      diff = cv2.absdiff(prev_gray, gray)
97                      _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
98
99                      # Dilate to fill gaps
100                     thresh = cv2.dilate(thresh, None, iterations=2)
101
102                     # Motion ratio
103                     motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
104                     motion_signals.append(motion_ratio)
105                 else:
106                     motion_signals.append(0.0)
107
108                 prev_gray = gray
109                 frame_idx += 1
110                 processed_count += 1
111
112                 if progress_step and frame_idx >= next_progress:
113                     pct = 100.0 * frame_idx / total_frames
114                     elapsed = time.time() - t_start
115                     eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
116                     logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
117                                 f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
118                     next_progress += progress_step
119
120                 if max_frames is not None and processed_count >= max_frames:
121                     logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
122                     break
123
124                 if frame_idx >= total_frames:
125                     break
126
127             cap.release()
128
129             # Smooth signal using a moving average
```

```
            window_size = 5
            smoothed = []
            for i in range(len(motion_signals)):
                start = max(0, i - window_size // 2)
                end = min(len(motion_signals), i + window_size // 2 + 1)
                smoothed.append(sum(motion_signals[start:end]) / (end - start))

            # Detect active play boundaries
            in_segment = False
            segment_start = None
            raw_segments: List[Tuple[float, float]] = []

            for i, t in enumerate(timestamps):
                signal = smoothed[i]
                if signal > motion_threshold:
                    if not in_segment:
                        in_segment = True
                        segment_start = t
                else:
                    if in_segment:
                        in_segment = False
                        raw_segments.append((segment_start, t))

            # Handle video ending while in active segment
            if in_segment:
                raw_segments.append((segment_start, timestamps[-1]))

            # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
            merged_segments: List[Tuple[float, float]] = []
            if raw_segments:
                curr_start, curr_end = raw_segments[0]
                for next_start, next_end in raw_segments[1:]:
                    if next_start - curr_end < dead_time_merge_gap:
                        # Merge
                        curr_end = next_end
                    else:
                        merged_segments.append((curr_start, curr_end))
                        curr_start, curr_end = next_start, next_end
                merged_segments.append((curr_start, curr_end))

            # Post-Processing: 2. Filter segments shorter than min_segment_duration
            final_segments: List[Tuple[float, float]] = [
                r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
            ]

            return final_segments

        def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
                                     player_pool: List[str]) -> List[Dict[str, Any]]:
            """Persist ``final_segments`` and their fabricated metadata/events to the DB.

            For each detected segment this assigns a random server/receiver, fabricates
            rich tags, then indexes a serve / 0?3 smash / termination event set ? all
            with the stdlib ``random`` module (the legacy demo's fabricated baseline).
            Returns the per-segment dicts handed back to the caller. The ``random``
            call order here is load-bearing for reproducibility and must not change.
            """
            # Populate SQLite DB with segments and metadata events
            segments_data = []
            for start, end in final_segments:
                # Assign Server and Receiver randomly from pool
                server, receiver = random.sample(player_pool, 2)

                # Formulate dynamic rich tags
```

```
                dur = end - start
                metadata = {
                    "shot_count": random.randint(4, 25),
                    "intensity": "high" if dur > 12 else "medium",
                    "avg_speed_kmh": round(random.uniform(40, 85), 1)
                }

                segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)

                if segment_id:
                    # Add sub-segment events
                    # A: Serve Event (always at start)
                    serve_foul = random.random() < 0.15 # 15% chance of serve foul
                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=start + 0.5,
                        event_type="serve",
                        player=server,
                        details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
                    )

                    # B: Smash Events (1 to 3 random smashes per segment)
                    smashes_count = random.randint(0, 3)
                    for _ in range(smashes_count):
                        smash_time = random.uniform(start + 1.5, end - 1.0)
                        smasher = random.choice([server, receiver])
                        self.db.add_event(
                            segment_id=segment_id,
                            timestamp=round(smash_time, 2),
                            event_type="smash",
                            player=smasher,
                            details={"speed_kmh": round(random.uniform(120, 250), 1)}
                        )

                    # C: Segment Termination Event (foul/winner)
                    foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
                    is_foul = random.random() > 0.4
                    error_player = random.choice([server, receiver])

                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=end - 0.2,
                        event_type="foul" if is_foul else "winner",
                        player=error_player,
                        details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
                    )

                    # Store segment representation to return to caller
                    segments_data.append({
                        "id": segment_id,
                        "start_time": start,
                        "end_time": end,
                        "duration": dur,
                        "server": server,
                        "receiver": receiver,
                        "metadata": metadata
                    })

            return segments_data

    # Register the segmenter
    segmenter_registry.register(MotionPlaySegmenter)
```

256

11. user (2026-06-22T09:12:24.597Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
```

```
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

12. assistant (2026-06-22T09:12:27.511Z)

Now I see a critical issue. Let me examine the random-based metadata generation more closely and check if there are other sources of non-determinism:

13. user (2026-06-22T09:12:29.923Z)

```
191:            server, receiver = random.sample(player_pool, 2)
196:                "shot_count": random.randint(4, 25),
198:                "avg_speed_kmh": round(random.uniform(40, 85), 1)
206:                serve_foul = random.random() < 0.15 # 15% chance of serve foul
212:                    details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
216:                smashes_count = random.randint(0, 3)
218:                    smash_time = random.uniform(start + 1.5, end - 1.0)
219:                    smasher = random.choice([server, receiver])
225:                        details={"speed_kmh": round(random.uniform(120, 250), 1)}
230:                is_foul = random.random() > 0.4
231:                error_player = random.choice([server, receiver])
238:                    details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
```

14. user (2026-06-22T09:12:31.020Z)

```
Found 2 files
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py
```

15. assistant (2026-06-22T09:12:35.115Z)

This is very significant. Let me check if there's random seed initialization anywhere, and also verify what the mapping said about counting play_segments:

16. user (2026-06-22T09:12:36.181Z)

```
No files found
```

17. assistant (2026-06-22T09:12:38.584Z)

Now let me verify the claim about segment count being deterministic by looking at the segment detection logic and the database:

18. user (2026-06-22T09:12:39.627Z)

```
450                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
451                 if play_after is not None:
452                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
453                 if cand_upto is not None:
454                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
455                 if cand_after is not None:
456                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457                 conn.commit()
458
```

```
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
462                     cursor = conn.cursor()
463                     cursor.execute(
464                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
466                     )
467                     conn.commit()
468                     return cursor.lastrowid
469             except Exception as e:
470                 logger.error(f"Failed to add play segment: {e}")
471                 return None
472
473         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474             return self._execute_write(
475                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477                 "Failed to add event",
478             )
479
480         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
482                                   stats: Optional[Dict[str, Any]] = None,
483                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
484             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
485             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
486             try:
487                 with self._connect() as conn:
488                     cursor = conn.cursor()
489                     cursor.execute(
490                         """INSERT INTO candidate_segments
491                            (video_id, detector, chunk_index, start_time, end_time, duration,
492                             confidence, stats, detection_params)
493                            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
494                         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
495                          confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
496                     )
497                     conn.commit()
498                     return cursor.lastrowid
499             except Exception as e:
```

19. user (2026-06-22T09:12:40.598Z)

```
1       import os
2       import shutil
3       import sqlite3
4       import json
5       import logging
6       from contextlib import contextmanager
7       from typing import List, Dict, Any, Optional, Tuple
8
9       from backend import billing  # pure cost math (no backend imports ? no cycle)
```

```
10
11      logger = logging.getLogger(__name__)
12
13      # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14      # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
15      # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16      # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17      # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18      SCHEMA_VERSION = 7
19
20
21      class SchemaTooNewError(RuntimeError):
22          """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24      class Database:
25          def __init__(self, db_path: str):
26              self.db_path = db_path
27              self._init_db()
28
29          @contextmanager
30          def _connect(self):
31              """Transaction scope that also CLOSES the connection.
32
33              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
34              never closes, so every call leaked its connection to GC (lingering file
35              handles on the WAL sidecars under Windows). Internal methods use this.
36              """
37              conn = self._get_connection()
38              try:
39                  with conn:
40                      yield conn
41              finally:
42                  conn.close()
43
44          def _get_connection(self):
45              # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
46              # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
47              conn = sqlite3.connect(self.db_path, timeout=30.0)
48              conn.row_factory = sqlite3.Row
49              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
50              conn.execute("PRAGMA foreign_keys = ON")
51              # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
52              conn.execute("PRAGMA busy_timeout = 30000")
53              try:
54                  conn.execute("PRAGMA journal_mode = WAL")
55              except sqlite3.OperationalError:
56                  pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
57              return conn
58
59          def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
60              """Run ONE bool-returning single-statement write inside a connection scope.
61
62              Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
63              the bool-returning single-statement writers. ``error_msg`` is the caller's exact
64              per-method message (already formatted with its own ids) ? logged verbatim on a
65              swallowed write fault so each writer keeps its specific log text. Returns True
on
66              success, False (after logging ``error_msg``) on any exception."""
```

```
67              try:
68                  with self._connect() as conn:
69                      conn.cursor().execute(sql, params)
70                      conn.commit()
71                  return True
72              except Exception as e:
73                  logger.error(f"{error_msg}: {e}")
74                  return False
75
76          @staticmethod
77          def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
78              """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
79
80              A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
```

20. assistant (2026-06-22T09:12:43.016Z)

Now let me check the key question: does the segment detection algorithm itself produce deterministic output (segment count and boundaries)?

21. user (2026-06-22T09:12:44.965Z)

```
39          def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40              """Run the pixel-motion heuristic over ``cap`` and return the detected
41              (start, end) play segments.
42
43              Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44              thresholds it into raw segments, then merges dead-time gaps and filters
45              out segments shorter than ``min_segment_duration``. ``cap`` is released
46              before returning. This is the detection half of ``process_video`` ? it
47              performs no DB writes and no fabrication.
48              """
49              fps = cap.get(cv2.CAP_PROP_FPS)
50              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51              duration = total_frames / fps if fps > 0 else 0
52
53              # Sub-sample settings (5 FPS for analysis)
54              analysis_fps = 5.0
55              frame_interval = max(1, int(fps / analysis_fps))
56
57              idx_cfg = self.config.get("indexing", {})
58              motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59              min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60              dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62              prev_gray = None
63              motion_signals = []
64              timestamps = []
65
66              frame_idx = 0
67              processed_count = 0
68              t_start = time.time()
69              # Emit a progress line roughly every 5% of the video so long runs are
70              # observable instead of silent (a full 1080p match is ~tens of minutes).
71              progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72              next_progress = progress_step
73              logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74                          f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75              while True:
76                  # Skip decoding for intermediate frames using cap.grab() for high
performance
77                  for _ in range(frame_interval - 1):
78                      if not cap.grab():
```

```
                        break
                    frame_idx += 1

                ret, frame = cap.read()
                if not ret:
                    break

                # Process frame
                gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                # Apply Gaussian Blur to smooth noise
                gray = cv2.GaussianBlur(gray, (21, 21), 0)

                t = frame_idx / fps
                timestamps.append(t)

                if prev_gray is not None:
                    # Frame absolute difference (highly optimized motion tracking)
                    diff = cv2.absdiff(prev_gray, gray)
                    _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

                    # Dilate to fill gaps
                    thresh = cv2.dilate(thresh, None, iterations=2)

                    # Motion ratio
                    motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                    motion_signals.append(motion_ratio)
                else:
                    motion_signals.append(0.0)

                prev_gray = gray
                frame_idx += 1
                processed_count += 1

                if progress_step and frame_idx >= next_progress:
                    pct = 100.0 * frame_idx / total_frames
                    elapsed = time.time() - t_start
                    eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
                    logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
                                f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
                    next_progress += progress_step

                if max_frames is not None and processed_count >= max_frames:
                    logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
                    break

                if frame_idx >= total_frames:
                    break

            cap.release()

            # Smooth signal using a moving average
            window_size = 5
            smoothed = []
            for i in range(len(motion_signals)):
                start = max(0, i - window_size // 2)
                end = min(len(motion_signals), i + window_size // 2 + 1)
                smoothed.append(sum(motion_signals[start:end]) / (end - start))

            # Detect active play boundaries
            in_segment = False
            segment_start = None
```

```
            raw_segments: List[Tuple[float, float]] = []

            for i, t in enumerate(timestamps):
                signal = smoothed[i]
                if signal > motion_threshold:
                    if not in_segment:
                        in_segment = True
                        segment_start = t
                else:
                    if in_segment:
                        in_segment = False
                        raw_segments.append((segment_start, t))

            # Handle video ending while in active segment
            if in_segment:
                raw_segments.append((segment_start, timestamps[-1]))

            # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
            merged_segments: List[Tuple[float, float]] = []
            if raw_segments:
                curr_start, curr_end = raw_segments[0]
                for next_start, next_end in raw_segments[1:]:
                    if next_start - curr_end < dead_time_merge_gap:
                        # Merge
                        curr_end = next_end
                    else:
                        merged_segments.append((curr_start, curr_end))
                        curr_start, curr_end = next_start, next_end
                merged_segments.append((curr_start, curr_end))

            # Post-Processing: 2. Filter segments shorter than min_segment_duration
            final_segments: List[Tuple[float, float]] = [
                r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
            ]

            return final_segments

        def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
                                     player_pool: List[str]) -> List[Dict[str, Any]]:
            """Persist ``final_segments`` and their fabricated metadata/events to the DB.

            For each detected segment this assigns a random server/receiver, fabricates
            rich tags, then indexes a serve / 0?3 smash / termination event set ? all
            with the stdlib ``random`` module (the legacy demo's fabricated baseline).
```

22. assistant (2026-06-22T09:12:47.599Z)

Perfect! Now I can see the structure clearly. Let me verify one final critical issue - does the database INSERT actually record the segments deterministically? Let me also check if there are any timing-dependent or floating-point issues:

23. user (2026-06-22T09:12:49.504Z)

```
import random
    def _detect_motion_segments(self, cap: Any, max_frames: int = None) -> List[Tuple[float,
float]]:
        t_start = time.time()
                elapsed = time.time() - t_start
        raw_segments: List[Tuple[float, float]] = []
        merged_segments: List[Tuple[float, float]] = []
        final_segments: List[Tuple[float, float]] = [
    def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float, float]],
        For each detected segment this assigns a random server/receiver, fabricates
        with the stdlib ``random`` module (the legacy demo's fabricated baseline).
```

```
        Returns the per-segment dicts handed back to the caller. The ``random``
            # Assign Server and Receiver randomly from pool
            server, receiver = random.sample(player_pool, 2)
                "shot_count": random.randint(4, 25),
                "avg_speed_kmh": round(random.uniform(40, 85), 1)
                serve_foul = random.random() < 0.15 # 15% chance of serve foul
                    details={"status": "foul" if serve_foul else "valid", "type": "low_serve" if
random.random() > 0.5 else "high_serve"}
                # B: Smash Events (1 to 3 random smashes per segment)
                smashes_count = random.randint(0, 3)
                    smash_time = random.uniform(start + 1.5, end - 1.0)
                    smasher = random.choice([server, receiver])
                        details={"speed_kmh": round(random.uniform(120, 250), 1)}
                is_foul = random.random() > 0.4
                error_player = random.choice([server, receiver])
                    details={"type": random.choice(foul_types)} if is_foul else {"shot_type":
"drop" if random.random() > 0.5 else "smash"}
```

24. assistant (2026-06-22T09:12:52.236Z)

Now let me verify the critical question: does the mapping's claim about COUNT of play_segments being deterministic hold true? Let me look at when segments are inserted:

25. user (2026-06-22T09:12:53.172Z)

```
177         def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
178                                      player_pool: List[str]) -> List[Dict[str, Any]]:
179             """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181             For each detected segment this assigns a random server/receiver, fabricates
182             rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183             with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184             Returns the per-segment dicts handed back to the caller. The ``random``
185             call order here is load-bearing for reproducibility and must not change.
186             """
187             # Populate SQLite DB with segments and metadata events
188             segments_data = []
189             for start, end in final_segments:
190                 # Assign Server and Receiver randomly from pool
191                 server, receiver = random.sample(player_pool, 2)
192
193                 # Formulate dynamic rich tags
194                 dur = end - start
195                 metadata = {
196                     "shot_count": random.randint(4, 25),
197                     "intensity": "high" if dur > 12 else "medium",
198                     "avg_speed_kmh": round(random.uniform(40, 85), 1)
199                 }
200
201                 segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
202
203                 if segment_id:
204                     # Add sub-segment events
205                     # A: Serve Event (always at start)
206                     serve_foul = random.random() < 0.15 # 15% chance of serve foul
207                     self.db.add_event(
208                         segment_id=segment_id,
209                         timestamp=start + 0.5,
210                         event_type="serve",
211                         player=server,
212                         details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
213                     )
```

```
214
215                     # B: Smash Events (1 to 3 random smashes per segment)
216                     smashes_count = random.randint(0, 3)
217                     for _ in range(smashes_count):
218                         smash_time = random.uniform(start + 1.5, end - 1.0)
219                         smasher = random.choice([server, receiver])
220                         self.db.add_event(
221                             segment_id=segment_id,
222                             timestamp=round(smash_time, 2),
223                             event_type="smash",
224                             player=smasher,
225                             details={"speed_kmh": round(random.uniform(120, 250), 1)}
226                         )
227
228                     # C: Segment Termination Event (foul/winner)
229                     foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
230                     is_foul = random.random() > 0.4
231                     error_player = random.choice([server, receiver])
232
233                     self.db.add_event(
234                         segment_id=segment_id,
235                         timestamp=end - 0.2,
236                         event_type="foul" if is_foul else "winner",
237                         player=error_player,
238                         details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
239                     )
240
241                     # Store segment representation to return to caller
242                     segments_data.append({
243                         "id": segment_id,
244                         "start_time": start,
245                         "end_time": end,
246                         "duration": dur,
247                         "server": server,
248                         "receiver": receiver,
249                         "metadata": metadata
250                     })
251
252             return segments_data
```

26. assistant (2026-06-22T09:12:55.279Z)

Now let me verify if there are any issues with add_play_segment that might affect the count:

27. user (2026-06-22T09:12:56.967Z)

```
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
462                     cursor = conn.cursor()
463                     cursor.execute(
464                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
466                     )
467                     conn.commit()
468                     return cursor.lastrowid
469             except Exception as e:
470                 logger.error(f"Failed to add play segment: {e}")
471                 return None
472
```

```
473         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474             return self._execute_write(
475                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477                 "Failed to add event",
478             )
479
480         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
482                                   stats: Optional[Dict[str, Any]] = None,
483                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
```

28. assistant (2026-06-22T09:12:59.687Z)

Now let me verify the claim about finalization and whether segments can be skipped or dropped:

29. user (2026-06-22T09:13:01.183Z)

```
278-    output_filename: str
279-    # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
280-    # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
281-    locale: Optional[str] = None
282-
283:def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
284-    """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
285-
286-    On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
287-    fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
288-    keep the prior good results intact ? a failed/empty re-run never destroys prior data.
289-    """
290-    if segments:
291:        db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
292-    else:
293:        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
294-
295-
296-# --- API Endpoints ---
297-@app.get("/api/config")
298-def get_config():
--
513-            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
514-        except Exception:
515-            # A raising segmenter must not leave half-written rows: restore the pre-run
516-            # state (same destroy-after-confirm contract as the Failure branch), then let
517-            # the job worker record the failure.
518:            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
519-            raise
520-        segmentation_ran = True
521-
522-        if isinstance(result, Failure):
523-            logger.error(f"[API] Segmenter failed: {result.message}")
524:            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
525-            response = {
526-                "video_id": video_id,
527-                "status": "failed",
528-                "message": f"Segmenter '{seg_name}' failed: {result.message}",
529-                "results": [r.model_dump() for r in val_results],
```

```
--
531-            }
532-            return response
533-
534-        segments = result.value if hasattr(result, "value") else result
535-        notify("finalizing")
536:        _finalize_reingest(video_id, segments, play_wm, cand_wm)
537-
538-        response = {
539-            "video_id": video_id,
540-            "status": "success",
541-            "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
--
1057-                segments = []
1058-            else:
1059-                segments = result.value if hasattr(result, "value") else result
1060-                print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}")
1061-            # Destroy-AFTER-confirm: keep new on success, restore prior on empty/failure.
1062:            _finalize_reingest(video_id, segments, play_wm, cand_wm)
1063-        finally:
1064-            # Always emit the per-video observability report. Never raises.
1065-            paths = emit_indexer_report(
1066-                db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
1067-                val_results=val_results, val_context=val_context,
```

30. assistant (2026-06-22T09:13:03.829Z)

Excellent. Now let me look for any floating-point precision or timing-dependent issues in segment detection:

31. user (2026-06-22T09:13:05.502Z)

```
3:import random
68:        t_start = time.time()
114:                elapsed = time.time() - t_start
181:        For each detected segment this assigns a random server/receiver, fabricates
183:        with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184:        Returns the per-segment dicts handed back to the caller. The ``random``
190:            # Assign Server and Receiver randomly from pool
191:            server, receiver = random.sample(player_pool, 2)
196:                "shot_count": random.randint(4, 25),
198:                "avg_speed_kmh": round(random.uniform(40, 85), 1)
206:                serve_foul = random.random() < 0.15 # 15% chance of serve foul
212:                    details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
215:                # B: Smash Events (1 to 3 random smashes per segment)
216:                smashes_count = random.randint(0, 3)
218:                    smash_time = random.uniform(start + 1.5, end - 1.0)
219:                    smasher = random.choice([server, receiver])
225:                        details={"speed_kmh": round(random.uniform(120, 250), 1)}
230:                is_foul = random.random() > 0.4
231:                error_player = random.choice([server, receiver])
238:                    details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
```

32. assistant (2026-06-22T09:13:08.012Z)

Now I need to check if there's any other source of non-determinism. Let me look at the compile path to see if segment count is truly stable:

33. user (2026-06-22T09:13:09.742Z)

```
687-    if not os.path.isfile(path):
688-        raise HTTPException(status_code=404,
689-                            detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
690-    media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
691-    return FileResponse(path, media_type=media, filename=name)
692-
693-
694-@app.post("/api/query")
695:def query_play_segments(req: QueryRequest):
696-    filters = {
697-        "server": req.server,
698-        "receiver": req.receiver,
699-        "duration_min": req.duration_min,
700-        "event_type": req.event_type
701-    }
702:    segments = db_instance.query_play_segments(req.video_id, filters)
703-    return {"play_segments": segments}
704-
705-@app.post("/api/compile")
706:def compile_highlights(req: CompileRequest):
707-    video = db_instance.get_video(req.video_id)
708-    if not video:
709-        raise HTTPException(status_code=404, detail="Indexed video record not found.")
710-
711-    # Retrieve matching play segments from db
712:    all_segments = db_instance.query_play_segments(req.video_id, {})
713-    matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
714-
715-    if not matched_segments:
716-        raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
717-
718-    # Reject path traversal / absolute output names (os.path.join would otherwise let an
719-    # absolute or '..'-laden output_filename write anywhere on disk).
720-    try:
--
757-                             end_padding=stitching.end_padding,
watermark=localized_watermark)
758-    try:
759-        # The stored `filepath` is the SOURCE ref ? a bucket\Drive URI (gs://, s3://,
gdrive://) for a
760-        # video ingested via /api/process from a URI, or a plain local path. Resolve it to a
LOCAL file
761-        # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
762-        # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
763-        # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a gs://
path.
764-        local_src = storage.acquire_local_input(video["filepath"], getattr(global_config,
"storage", None))
765:        out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
766-        # `filepath` is server-local (not reachable by the browser); also return the ready-
to-use
767-        # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
768-        download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
769-        return {"status": "success", "filepath": out_path, "download_url": download_url}
770-    except Exception as e:
771-        raise HTTPException(status_code=500, detail=f"Highlight stitching failed: {str(e)}")
from e
772-
773-# --- CLI Debug Utility & Orchestration ---
--
917-
```

```
918-        # global_config isn't initialized this early; load the typed config so the trim
919-        # clip carries the same configured stitching paddings + watermark as a real
920-        # highlight (watermark default-on).
921-        stitching = load_app_config().stitching
922-        compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
923-                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
924-        try:
925:            res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
926-            print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
927-        except Exception as e:
928-            print(f"[ERROR] Failed to compile trimmed segment: {e}")
929-        return
930-
931-    if args.debug_clip is not None:
932-        if not args.video_path:
933-            print("[ERROR] Please provide --video-path to debug a specific clip.")
```

34. assistant (2026-06-22T09:13:12.594Z)

Now let me verify my critical finding - that add_play_segment CAN fail silently (returns None) but the iteration continues:

35. user (2026-06-22T09:13:14.647Z)

```
187             # Populate SQLite DB with segments and metadata events
188             segments_data = []
189             for start, end in final_segments:
190                 # Assign Server and Receiver randomly from pool
191                 server, receiver = random.sample(player_pool, 2)
192
193                 # Formulate dynamic rich tags
194                 dur = end - start
195                 metadata = {
196                     "shot_count": random.randint(4, 25),
197                     "intensity": "high" if dur > 12 else "medium",
198                     "avg_speed_kmh": round(random.uniform(40, 85), 1)
199                 }
200
201                 segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
202
203                 if segment_id:
204                     # Add sub-segment events
205                     # A: Serve Event (always at start)
206                     serve_foul = random.random() < 0.15 # 15% chance of serve foul
207                     self.db.add_event(
208                         segment_id=segment_id,
209                         timestamp=start + 0.5,
210                         event_type="serve",
211                         player=server,
212                         details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
213                     )
214
215                     # B: Smash Events (1 to 3 random smashes per segment)
216                     smashes_count = random.randint(0, 3)
217                     for _ in range(smashes_count):
218                         smash_time = random.uniform(start + 1.5, end - 1.0)
219                         smasher = random.choice([server, receiver])
220                         self.db.add_event(
221                             segment_id=segment_id,
222                             timestamp=round(smash_time, 2),
223                             event_type="smash",
```

```
224                             player=smasher,
225                             details={"speed_kmh": round(random.uniform(120, 250), 1)}
226                         )
227
228                     # C: Segment Termination Event (foul/winner)
229                     foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
230                     is_foul = random.random() > 0.4
231                     error_player = random.choice([server, receiver])
232
233                     self.db.add_event(
234                         segment_id=segment_id,
235                         timestamp=end - 0.2,
236                         event_type="foul" if is_foul else "winner",
237                         player=error_player,
238                         details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
239                     )
240
241                     # Store segment representation to return to caller
242                     segments_data.append({
243                         "id": segment_id,
244                         "start_time": start,
245                         "end_time": end,
246                         "duration": dur,
247                         "server": server,
248                         "receiver": receiver,
249                         "metadata": metadata
250                     })
251
```

36. assistant (2026-06-22T09:13:17.086Z)

Perfect! I found a critical issue. Let me now verify my understanding by checking the exact flow one more time and see if there are any other sources of non-determinism:

37. user (2026-06-22T09:13:22.309Z)

MotionPlaySegmenter._detect_motion_segments signature:
(self, cap: Any, max_frames: int = None) -> List[Tuple[float, float]]

38. assistant (2026-06-22T09:13:24.343Z)

Excellent. Now let me check the exact commitment that the mapping made:

39. user (2026-06-22T09:13:25.317Z)

```
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
462                     cursor = conn.cursor()
463                     cursor.execute(
464                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
466                     )
467                     conn.commit()
468                     return cursor.lastrowid
469             except Exception as e:
470                 logger.error(f"Failed to add play segment: {e}")
471                 return None
472
```

```
473         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
```

40. assistant (2026-06-22T09:13:27.310Z)

Now let me verify that the comment in motion.py line 185 is correct:

41. user (2026-06-22T09:13:28.266Z)

```
179             """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181             For each detected segment this assigns a random server/receiver, fabricates
182             rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183             with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184             Returns the per-segment dicts handed back to the caller. The ``random``
185             call order here is load-bearing for reproducibility and must not change.
186             """
187             # Populate SQLite DB with segments and metadata events
188             segments_data = []
```

42. assistant (2026-06-22T09:13:49.307Z)

Perfect! I now have all the evidence I need. Let me compile my findings:

Summary of Evidence

The mapping claims: **"The COUNT of reels/highlight-segments produced by the configured+checkpointed pipeline is DETERMINISTIC for a fixed (video, config, weights) ? safe to assert exactly (==), not just approximately."**

Evidence of DETERMINISM (Supporting the claim):

1. **Segment DETECTION is deterministic:** The `_detect_motion_segments()` method (motion.py:39-175) is purely deterministic:
   - Reads video frames sequentially (no randomness in frame order)
   - Applies deterministic CV operations: `cv2.absdiff()`, `cv2.threshold()`, `cv2.GaussianBlur()`, `cv2.dilate()`, `cv2.countNonZero()`
   - Applies deterministic signal smoothing (moving average, lines 129-135)
   - Applies deterministic thresholding (lines 142-155) based on config values (`motion_threshold`, `min_segment_duration`, `dead_time_merge_gap`)
   - Returns a list of (start_time, end_time) tuples ? NO randomness

2. **Segment INSERTION follows the detected count:** Line 189 loops `for start, end in final_segments:` ? the COUNT of iterations equals the COUNT from `_detect_motion_segments()`.

3. **The final segments list is deterministic:** The mapping explicitly notes that `_detect_motion_segments()` "performs no DB writes and no fabrication" ? it is pure.

Evidence of NON-DETERMINISM (Refuting the claim):

CRITICAL FINDING: The `_populate_db_with_events()` method (motion.py:177-252) uses **unseeded `random` module calls** INSIDE the per-segment loop:

- Line 191: `server, receiver = random.sample(player_pool, 2)` ? random assignment
- Line 196: `"shot_count": random.randint(4, 25)` ? random metadata
- Lines 206, 212, 216, 218, 219, 225, 230, 231, 238: **MULTIPLE `random.*()` calls per segment**

Critical issue at line 201-203:
```python
segment_id = self.db.add_play_segment(video_id, start, end, server, receiver, metadata)

if segment_id:
    # Add sub-segment events
```

The `add_play_segment()` (database.py:459-471) **CAN FAIL AND RETURN `None`** silently:
```python
except Exception as e:
    logger.error(f"Failed to add play segment: {e}")
    return None
```

If any call to `add_play_segment()` throws an exception (DB locked, I/O error, constraint violation, etc.) **the segment_id will be None**, the `if segment_id:` block SKIPS adding events, but execution CONTINUES.

Result: The COUNT of rows in `play_segments` table can vary between runs if ANY database operation fails, even transiently (e.g., disk full, lock timeout, WAL contention).

Secondary issue:

The comment at motion.py:185 states: *"The `random` call order here is load-bearing for reproducibility and must not change."*

This acknowledges that **reproducibility depends on random**, which contradicts the claim that COUNT is deterministic. The comment implies metadata fields (shot_count, intensity, etc.) are pseudo-reproducible only if random is seeded externally ? **but there is NO seed initialization in the codebase** (Grep found zero matches for `random.seed`, `np.random.seed`, or `set_seed`).

Final verdict:

The **segment detection algorithm itself IS deterministic**. However, the **play_segments TABLE COUNT is NOT deterministic** because:

1. Database INSERT can fail silently, causing segments to be skipped
2. There is no error handling to distinguish "no segments" from "partial DB failure"
3. Transient DB locks (especially under Windows WAL with concurrent readers/writers per database.py:46-47 comment) could cause non-deterministic failures

The claim is REFUTED with high confidence.

43. user (2026-06-22T09:13:57.805Z)

Structured output provided successfully